



记忆化搜索

搜索，实际上就是一种尝试所有可能性的做法，但是实际上我们需要的最优的结果一般只有一个，所以在搜索过程中，其中大部分尝试都是错误的，故此搜索的效率比较低，所以我们要做的就是提升搜索的效率，也就是我们常说的剪枝，如果该种方案不可能是最优解，那么我们就提前结束，不再往下尝试。

剪枝的方法有很多种，这里介绍一种效率比较高的方法——记忆化搜索。当然，并不是所有的搜索都可以用记忆化搜索来实现。



斐波那契数列

已知斐波那契数列 $f_1=1$, $f_2=1$, 求斐波那契数列第 n 项是多少

该题可以直接递推, $f_3=f_1+f_2$, $f_4=f_3+f_2$, 如果我们要用递归来实现呢?

```
int f(int x)
{
    if(x==1)
        return 1;
    if(x==2)
        return 1;
    return f(x-1)+f(x-2);
}
```

斐波那契数列

但是我们来调试一下会怎么样呢？输入10，很快输出55，接下来输入1000，发现很久都没有出来结果？为什么会这样呢？是因为递归的效率太低了吗？

我们来手推一下，如果要计算第10项的结果，该递归函数会被调用多少次。

实际上我们发现，该函数被调用的次数特别多，差不多是2的阶乘次，所以当我们输入100的时候，结果根本出不来。那有没有办法可以让递归的效率跟高一点呢？和递推的效率达到一样。

斐波那契数列

我们发现函数调用的过程中，效率低的原因在于当我们递归计算出 f_8 的值之后，再去计算 f_9 的值得时候，把 f_9 分解为 f_8+f_7 ，然后继续去分解 f_8 ，但是实际上在前面我们已经在 f_8 的结果得到了，实际上并不需要我们去递归了。同样，下面的 $f_7, f_6 \cdots f_1$ 都是一样被多次重复计算，这样就导致了效率大大降低。

那我们应该如何优化呢？

很简单，当我们计算出一项值得时候，直接开一个数组保存，当后面又调用到相同的项时，直接返回该值，而不是继续递归调用函数。

斐波那契数列

```
int f(int x)
{
    if(fa[x])//如果该项值已经被求出来了
        return fa[x];
    if(x==1)
        return f[1]=1;
    if(x==2)
        return f[2]=1;
    return fa[x]=f(x-1)+f(x-2);//把得到的返回值赋值给数组
}
```

这样，每一项都只会被计算一次，最多再被判断一次，所以时间复杂度还是 $O(n)$ 的，和递推的效率是一样的。



记忆化搜索

记忆化搜索的本质就是，当你搜索到一种状态的时候，如果该状态到最终状态的真实解我们已经得到了，那么我们就没有必要再继续搜索下去了，而是直接调用我们前面搜索到已经保存下来的答案。

但是记忆化搜索的时候要注意，一定要保证得到的某一种状态到最终状态的结果一定是真实答案，也就是正解，否则不可以用记忆化搜索。



滑雪

题目描述

[展开](#)

Michael喜欢滑雪。这并不奇怪，因为滑雪的确很刺激。可是为了获得速度，滑的区域必须向下倾斜，而且当你滑到坡底，你不得不再次走上坡或者等待升降机来载你。Michael想知道在一个区域中最长的滑坡。区域由一个二维数组给出。数组的每个数字代表点的高度。下面是一个例子：

```
1   2   3   4   5
16  17  18  19   6
15  24  25  20   7
14  23  22  21   8
13  12  11  10   9
```

一个人可以从某个点滑向上下左右相邻四个点之一，当且仅当高度减小。在上面的例子中，一条可行的滑坡为24 - 17 - 16 - 1（从24开始，在1结束）。当然25 - 24 - 23 - ... - 3 - 2 - 1更长。事实上，这是最长的一条。

输入格式

输入的第一行为表示区域的二维数组的行数R和列数C（ $1 \leq R, C \leq 100$ ）。下面是R行，每行有C个数，代表高度(两个数字之间用1个空格间隔)。

输出格式

输出区域中最长滑坡的长度。

滑雪

该题没有限制起点，所以从任意一个点作为起点都可以，并且并不是高度越高，滑得就越远，而是和周围的地形有关，所以我们要枚举起点，然后再搜索一条合理的路径。

但是这样时间复杂度为 $n*n*[搜索时间复杂度]$ ，肯定会超时，我们同样可以考虑记忆化，当我们从一个点出发得到一条从该点出发的最短路径，那么得到的这条最短路肯定是正确的。那么当我从其他点到达该点的时候，既然从该点出发的最短路我们已经知道了，那么我么就没有必要再搜索下去了，直接调用就好，并且因为是从高往低滑，所以这两条路径也是没有重复的。

滑雪

```
if(dis[x][y])//记忆化搜索, 如果以前找到了该位置的最长路径就直接返回
return dis[x][y];
int flag=0;
for(int k=0;k<4;k++)
{
    int tx=x+nextn[k][0];
    int ty=y+nextn[k][1];
    if(tx<1||ty<1||tx>n||ty>m)
        continue;
    if(map[x][y]>map[tx][ty])
    {
        flag=1;
        dis[x][y]=max(dis[x][y],dfs(tx,ty)+1);//记录该点的最长路径
    }
}
if(flag==0)//表示四周都没有比他低的点
dis[x][y]=1;
return dis[x][y];
```



最长路

对于一个有向无环图，求出从一个入度为0起点到一个出度为0的终点的路径之和最大。



最长路

对于DAG图，一般都是用Tp排序+dp来做，实际上我们也可以用记忆化搜索。

因为是有向无环图，所以不需要担心路径会重复，所以我们可以先从任意一个点出发搜索出所有的终点，以及路径上任意一个点到终点的路径最大长度是多少。当下一次，换起点的时候，如果搜到一个点 i ，并且这个点被访问过，那么后面就没有必要在搜索下去了，直接返回以前的dp值，效率和tp排序是一样的。

最长路

```
if(!cd[u])//搜到终点
{
    visit[u]=1;
    dp[u]=w[u];
    return dp[u];//记录到终点的点权
}
if(visit[u])
return dp[u];//直接返回
long long ans=-inf;
for(int i=head[u];i;i=edge[i].next)
{
    int v=edge[i].to;
    ans=max(ans,dfs(v)+w[u]);//取最大值
}
dp[u]=ans;
visit[u]=1;
return dp[u];
```

分割矩阵

题目描述

展开

将一个 $a*b$ 的数字矩阵进行如下分割：将原矩阵沿某一条直线分割成两个矩阵，再将生成的两个矩阵继续如此分割（当然也可以只分割其中的一个），这样分割了 $(n-1)$ 次后，原矩阵被分割成了 n 个矩阵。（每次分割都只能沿着数字间的缝隙进行）

原矩阵中每一位置上有一个分值，一个矩阵的总分为其所含各位置上分值之和。现在需要把矩阵按上述规则分割成 n 个矩阵，并使各矩阵总分的均方差最小。

请编程对给出的矩阵及 n ，求出均方差的最小值。

输入格式

第一行为3个整数，表示 a, b, n ($1 < a, b \leq 10, 1 < n \leq 10$) 的值。

第二行至第 $n+1$ 行每行为 b 个小于100的非负整数，表示矩阵中相应位置上的分值。每行相邻两数之间用一个空格分开。

输出格式

仅一个数，为均方差的最小值（四舍五入精确到小数点后2位）

分割矩阵

该题很容易猜到是一道动态规划的题，但是如果我们的动态规划的水平并不高，写不出来状态转移方程，应该怎么办呢？

那很明显就是写爆搜。

对于当前的一个矩形 (a, b, c, d) ，分割的时候可以横着分割，割线范围 $[a, c)$ （分成 (a, b, i, d) 和 $(i+1, b, c, d)$ ），也可以竖着分割，割线范围 $[b, d)$ （分成 (a, b, c, i) 和 $(a, i+1, c, d)$ ），那么爆搜就写出来了。

对于当前的一个矩形 (a, b, c, d) ，需要隔成 k 个矩形，可以先分一次，分成两个子矩形，然后在让两个子矩形分割成的矩形数量之和 $= k$ 。

分割矩阵

```
for(int i=a;i<c;i++)//如果横着切
{
    for(int j=1;j<num;j++)//其中一个矩形分割成j块, 另一个num-j块
    {
        double tmp=dfs(a,b,i,d,j)+dfs(i+1,b,c,d,num-j);
        dp[a][b][c][d][num]=min(dp[a][b][c][d][num],tmp);
    }
}
for(int i=b;i<d;i++)//如果竖着切
{
    for(int j=1;j<num;j++)
    {
        double tmp=dfs(a,b,c,i,j)+dfs(a,i+1,c,d,num-j);
        dp[a][b][c][d][num]=min(dp[a][b][c][d][num],tmp);
    }
}
```

分割矩阵

既然是搜索那么肯定要剪枝，如果同样的一块矩形 (a, b, c, d) 被分割成 k 块，那么肯定取搜索到的最小的答案，所以我们可以记录一个 $dp[a][b][c][d][k]$ 表示左上角顶点为 (a, b) ，右下角顶点为 (c, d) 的矩形被分割成 k 块的最小值。所以我们可以记忆化。

```
if(dp[a][b][c][d][num])
return dp[a][b][c][d][num];
if(num==1)//分割成一个矩阵的值可以直接算出来
{
    int tmp=js(a,b,c,d);
    return (tmp-ave)*(tmp-ave);
}
```

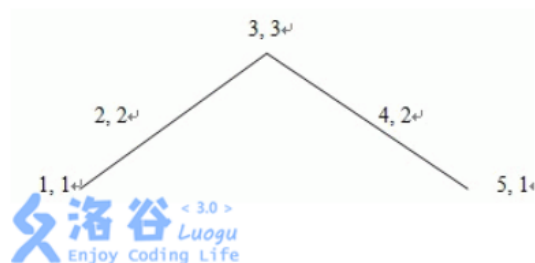

天天酷跑

题目描述

展开

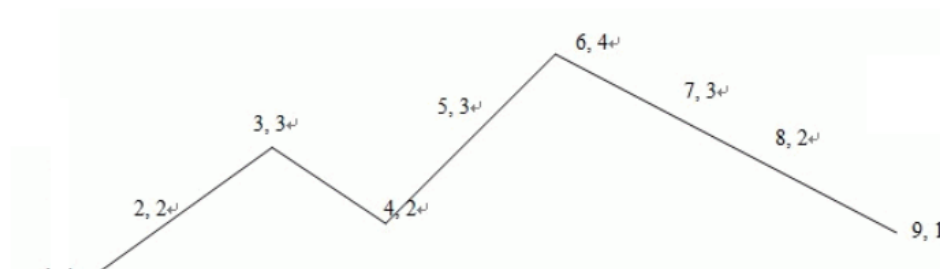
在游戏天天酷跑中，最爽的应该是超级奖励模式了吧，没有一切障碍，可以尽情的吃金币，现在请你控制游戏角色来获得尽可能多的分数。游戏界面离散为一个长度为 $1 \sim n$ ，高度为 $1 \sim m$ （初始点为 $(0, 1)$ ）的矩阵图。每个格子上都有收益 $(-1 \sim 1000)$ ， -1 表示该点不能通过。游戏角色从起点一路奔跑向终点，中途可以跳跃来获得更高的分数，在空中还能进行连跳。

游戏开始前你可以设定跳跃的高度，以及能连跳的次数，初始跳跃高度为1，连跳数为1（最多为5），升级跳跃高度和连跳都需要一定的花费。跳跃高度设定完后游戏角色每次跳跃高度都将固定，连跳必须在下落过程中可以使用。所有操作都将在整点上完成，需要保证设定完的跳跃高度及连跳数，无法跳出游戏高度上限。



从 $(1, 1)$ 点用一次跳跃，一次经过 $(2, 2), (3, 3), (4, 2), (5, 1)$ 。

以下是连跳数为2连跳，跳跃高度为2的跳跃方案：



天天酷跑

该题和上一道题实际上是一样的，因为每次跳跃的高度和连续跳跃的次数并没有单调性，所以直接枚举就可以了，对于枚举的每一种情况，去搜索。

对于当前位置 (x, y) ，如果最多还可以跳跃 k 次，如果前面已经得到了这个结果，那么就可以直接使用。所以我们记录一下 $dp[x][y][k]$ 。

对于当前这个状态，后续有三种可能，如果还可以跳跃，那么可以跳，如果不可以跳跃，如果已经在地面上了，那么久平移，如果在空中，就往下掉，最多就这三种情况，然后对答案取一个最大值，并把它赋值给 $dp[x][y][k]$ 就可以了。

注意边界，以及行列和坐标的区别。

玩具取名

Description

某人有一套玩具，并想法给玩具命名。首先他选择WING四个字母中的任意一个字母作为玩具的基本名字。然后他会根据自己的喜好，将名字中任意一个字母用“WING”中任意两个字母代替，使得自己的名字能够扩充得很长。现在，他想请你猜猜某一个很长的名字，最初可能是由哪几个字母变形过来的。

Input

第一行四个整数W、I、N、G。表示每一个字母能由几种两个字母所替代。接下来W行，每行两个字母,表示W可以用这两个字母替代。接下来I行，每行两个字母,表示I可以用这两个字母替代。接下来N行，每行两个字母,表示N可以用这两个字母替代。接下来G行，每行两个字母,表示G可以用这两个字母替代。最后一行一个长度不超过Len的字符串。表示这个玩具的名字。

Output

一行字符串，该名字可能由哪些字母变形而得到。（按照WING的顺序输出）如果给的名字不能由任何一个字母变形而得到则输出“The name is wrong!”

玩具取名

该题是一道dp题，但是如果你的DP比较水，一遇到dp脑袋就懵了，你不妨假设他就是一道搜索题。

对于一个最终的字符串，问可以由多少种初始字符串转换而来，那么我们就换一种思路，把目标字符串当做初始字符串，对调交换规则，如 $A \rightarrow BC$ ，改为 $BC \rightarrow A$ 。

对于一个初始字符串，只有当他长度只有一个的时候才是不变换的，那么怎么缩短呢？很明显是分治思想，把一个字符串拆成两个。怎么拆分最优呢？我们肯定事先不知道，所以枚举拆的中点呗？

同样，对于同一个区间不同的拆分方法最终得到的一个字符也不一样，但是也就四种：W, I, N, G

玩具取名

我们按照不同的拆分方法，对于区间 $[l, r]$ 可能都会拆分得到字符 k ，如果前面已经得到了这种可能性是成立的，那么如果按照其它方式拆分，也拆分到区间 $[l, r]$ 的时候，问是否可以得到字符 k ，我们完全不需要再去继续拆分到只有一个字符串为止了，而是直接返回结果。

玩具取名

```
bool dp(int l,int r,int k)
{
    if(l==r)//区间长度为一，不可以拆分，直接判断是否相等
    return (ch[l]==p[k]);
    if(f[l][r][k]!=-1);
    return f[l][r][k];//b表示区间[l,r]是否可以变换成字符k
    for(int i=1;i<=t[k];i++)//枚举变换规则
        for(int j=l;j<=r-1;j++)//枚举区间断点
            if(dp(l,j,q[a[k][i][0]])&&dp(j+1,r,q[a[k][i][1]]))
                //左区间=第一个字符，右区间=第二个字符
                return f[l][r][k]=1;
    return res=0;
}
```

玩具取名

实际上你会发现，记忆化搜索和动态规划的效率在很多情况下是相等的，很多时候一道记忆化搜索的题可以转换成动态规划来写，同样一道动态规划的题也可以转换成记忆化搜索来写。

相对动态规划而言，我们是从最简单状态推导到最终的状态，而记忆化搜索是我们知道最终状态之后，去判断该状态可以由哪些状态到达，再判断这些状态又可以由哪些状态到达，直接当前状态是我们已知的状态我们就可以直接把结果返回上一层。

一般来说，动态规划时间复杂度常数很小，而记忆化搜索常数会大一些，但是记忆化搜索是搜状态，搜到为止状态就继续搜，搜到已知状态就返回，不怎么需要考虑边界，转移方向等等。

关路灯

题目描述

展开

某一村庄在一条路线上安装了 n 盏路灯，每盏灯的功率有大有小（即同一段时间内消耗的电量有多有少）。老张就住在这条路中间某一路灯旁，他有一项工作就是每天早上天亮时一盏一盏地关掉这些路灯。

为了给村里节省电费，老张记录下了每盏路灯的位置和功率，他每次关灯时也都是尽快地去关，但是老张不知道怎样去关灯才能够最节省电。他每天都是在天亮时首先关掉自己所处位置的路灯，然后可以向左也可以向右去关灯。开始他以为先算一下左边路灯的总功率再算一下右边路灯的总功率，然后选择先关掉功率大的一边，再回过头来关掉另一边的路灯，而事实并非如此，因为在关的过程中适当地调头有可能会更省一些。

现在已知老张走的速度为 1m/s ，每个路灯的位置（是一个整数，即距路线起点的距离，单位： m ）、功率（ W ），老张关灯所用的时间很短而可以忽略不计。

请你为老张编一程序来安排关灯的顺序，使从老张开始关灯时刻算起所有灯消耗电最少（灯关掉后便不再消耗电了）。

输入格式

文件第一行是两个数字 n （ $1 \leq n \leq 50$ ，表示路灯的总数）和 c （ $1 \leq c \leq n$ 老张所处位置的路灯号）；

接下来 n 行，每行两个数据，表示第1盏到第 n 盏路灯的位置和功率。数据保证路灯位置单调递增。

输出格式

一个数据，即最少的功耗（单位： J ， $1\text{J} = 1\text{W}\cdot\text{s}$ ）。

关路灯

对于每一个位置，就两种选择，要么去关左边第一盏还未关掉的灯，要么去关右边第一盏还没关的灯。

```
void dfs(int id, int t, int sum, int l, int r)
{
    if (sum >= Min)
        return ;
    if (l == 0 && r == n + 1) // 已经关完了
    {
        Min = sum;
        return ;
    }
    if (l != 0) // 如果可以向左关灯
    {
        int tmp = a[id] - a[l];
        dfs(l, t + tmp, sum + (t + tmp) * b[l], l - 1, r);
    }
    if (r != n + 1)
    {
        int tmp = a[r] - a[id];
        dfs(r, t + tmp, sum + (t + tmp) * b[r], l, r + 1);
    }
    return ;
}
```

关路灯

在关灯的时候肯定是从初始位置向两边关灯，所以是不会出现重复的情况的，所以我们完全可以记录一下关掉区间 $[l, r]$ 的所花的时间，如果下次搜索到该位置的时候，要么更新答案，要么剪枝。

```
if(dp[l][r]<=sum)
return ;
dp[l][r]=sum;
if(l==0&&r==n+1)//已经关完了
return ;
if(l!=0)//如果可以向左关灯
{
    int tmp=a[id]-a[l];
    dfs(l, t+tmp, sum+(t+tmp)*b[l], l-1, r);
}
if(r!=n+1)
{
    int tmp=a[r]-a[id];
    dfs(r, t+tmp, sum+(t+tmp)*b[r], l, r+1);
}
```

关路灯

但是这样做有一点问题，因为当你关完区间 $[l, r]$ 的灯之后，你所处的位置和答案是有关系的，所以只有 $dp[l][r]$ 的答案是错误的，应该还要加上一维0/1表示此时处于左端点还是右端点。

```
if(dp[l][r][id]<=sum)
    return ;
dp[l][r][id]=sum;
if(l==0&&r==n+1)//已经关完了
    return ;
if(l!=0)//如果可以向左关灯
{
    int tmp=a[id]-a[l];
    dfs(l, t+tmp, sum+(t+tmp)*b[l], l-1, r);
}
if(r!=n+1)
{
    int tmp=a[r]-a[id];
    dfs(r, t+tmp, sum+(t+tmp)*b[r], l, r+1);
}
```

关路灯

但是提交上去之后就会发现，还是会错一个点。因为我们只考虑了当前功耗最小，却忽略了时间对后续关灯的影响。可能会出现这样两种情况，对于区间 $[l, r]$ ，第一种，关灯花的时间为 t_1 ，消耗功率为 w_1 ，第二种，关灯花的时间为 t_2 ，消耗功率为 w_2 ，如果出现 $w_1 < w_2$ 但是 $t_1 > t_2$ 这种情况，我们不能直接选前一种，因为如果 t 比较大，那么后面的灯消耗的功率也比较大。所以该题我们还需要考虑后效性。也就是在该时间差的基础上，后面灯多消耗的功率也应该考虑进去，一起做比较，得到的答案才是正确的。

关路灯

但是提交上去之后就会发现，还是会错一个点。因为我们只考虑了当前功耗最小，却忽略了时间对后续关灯的影响。可能会出现这样两种情况，对于区间 $[l, r]$ ，第一种，关灯花的时间为 t_1 ，消耗功率为 w_1 ，第二种，关灯花的时间为 t_2 ，消耗功率为 w_2 ，如果出现 $w_1 < w_2$ 但是 $t_1 > t_2$ 这种情况，我们不能直接选前一种，因为如果 t 比较大，那么后面的灯消耗的功率也比较大。所以该题我们还需要考虑后效性。也就是在该时间差的基础上，后面灯多消耗的功率也应该考虑进去，一起做比较，得到的答案才是正确的。这是一种考虑后效性的方法。

```
int s=js(1,1)+js(r,n);
if(dp[l][r][id]+f[l][r][id]*s<=sum+t*s)
return ;
dp[l][r][id]=sum;
f[l][r][id]=t;//要同时记录时间和功率，两者之差最小才是正解
```



练习题

洛谷 P1807 最长路

洛谷 P2217 分割矩阵

洛谷 P3257 天天酷跑

洛谷 P4290 玩具取名

洛谷 P1220 关路灯

洛谷 P4338 道路

洛谷 P2476 着色方案

